Prime Number Testing Methods

1. Trial Division Method

The Trial Division method is one of the crucial but one of the easiest factorization techniques when dealing with integer factorization.

Firstly, the naive approach of it. Since a prime number is only divided by itself and 1, we can loop on all numbers starting from 2 till n - 1 (n is the number we are testing its primality) to check if n is divisible by any of them, if not then it's a prime number, This approach has O(n-2) approx. O(n) time complexity.

Secondly, The Trial Division Method. We observe from the naive approach that the maximum factor of any number is less than or equal to $\sqrt{n}$ or n/2.

Proof [1] [2]**:**

Note: A composite number is a positive integer greater than 1 that has more than two factors. (Not Prime)

1. Assume n is composite number, it can be expressed as a product of 2 numbers, n = a × b, where a > 1 and b > 1
2. For any pair of factors a and b there are 3 possibilities:
   - Both are greater than $\sqrt{n}$
   - Both are less than $\sqrt{n}$
   - One is greater than $\sqrt{n}$ and the other is smaller
3. If both are greater than $\sqrt{n}$ and multiplying them together gives us this inequality: a × b > $\sqrt{n}$ × $\sqrt{n}$ which simplifies to: a × b > n, this contradicts the original statement n = a × b, So this scenario is impossible.
4. Since the case where both factors are greater than $\sqrt{n}$ is impossible, at least one of the factors must be less than or equal to $\sqrt{n}$
5. Hence, the largest possible factor for n is $\sqrt{n}$.

From this proof we can optimize the naive approach instead of looping on all number from 2 to n-1, we can loop till $\sqrt{n}$, If you find no divisors in this range, the number must be prime. This approach has O($\sqrt{n}$) instead of O(n).

---

[1] Geeks For Geeks "Trial Division Algorithm for Prime Factorization"
https://www.geeksforgeeks.org/dsa/trial-division-algorithm-for-prime-factorization/

[2] De Cuester Academy "4.3.1 Discrete Math - Primes and Trial Division"
https://www.youtube.com/watch?v=FJTHj0kG4hg&t=620s

2. Sieve of Eratosthenes [3]

The Sieve of Eratosthenes is one of the oldest and most efficient algorithms for generating all prime numbers up to a given number n. Unlike Trial Division, which checks the primality of a single number, the sieve is designed to find all primes within a range in an optimized way.

Steps:

1. Create a list of consecutive integers from 2 to n.
2. Start with the first prime number 2.
3. Mark all multiples of 2 (except 2 itself) as composite.
4. Move to the next unmarked number (which will be prime).
5. Repeat the process for its multiples.
6. Stop when you reach $\sqrt{n}$, because any composite number less than or equal n must have a factor less than or equal $\sqrt{n}$.

Example: Find all prime numbers up to 30.

1. Start with a list: 2, 3, 4, ..., 30
2. First prime = 2 → mark 4, 6, 8, 10, ... as composite
3. Next unmarked number = 3 → mark 6, 9, 12, 15, ...
4. Next unmarked = 5 → mark 10, 15, 20, 25, ...
5. Once we reach primes greater than $\sqrt{30}$ (~5.47), we stop marking.

Remaining unmarked numbers:
2, 3, 5, 7, 11, 13, 17, 19, 23, 29 → These are the primes.

Time complexity:

Each prime p marks its multiples which is nearly n/p. Ex: 30/2 = 15, so prime number 2 marks 15 of its multiples. Summing this over all primes gives $\sum_{p \le n} \frac{n}{p}$

$= n \sum_{p \le n} \frac{1}{p} \approx n$ (log (log n) + C). (Known mathematical result from number theory).

The total time is proportional to n × (1/2 + 1/3 + 1/5 + ...)

So , time complexity is O(n log (log n))

[3] Geeks for Geeks "Sieve of Eratosthenes" https://www.geeksforgeeks.org/dsa/sieve-of-eratosthenes/

3. Fermat Primality Test [4] [5] [6]

The Fermat Primality Test is a probabilistic algorithm based on ***Fermat's Little Theorem****, which gives a quick method for checking whether a number is likely to be prime. Unlike Trial Division or the Sieve of Eratosthenes, this test does* ***not*** *guarantee correctness but is extremely fast in practice.*

Fermat's Little Theorem states that if p is prime and a is not divisible by p, then

$a^{p-1} \equiv 1 \pmod{p}$ for any integer a such that gcd(a,p)=1 (gcd short for Greatest common divisor)

In simpler terms and steps:

1. Repeat the following k times (iterating)
   - Choose number a where 1 < a < n - 1 (n is the number we are testing)
   - Calculate $a^{n-1} \bmod n$, if it's not = 1, n is definitely not prime.
   - If it's equal to 1, then n is probable prime (continue in iterations to make sure more and more)
2. If it passes all tests, return **probably prime**

Increasing k makes the test more reliable.

Time Complexity:

Main operation is $a^{n-1} \bmod n$ which is done using fast modular exponentiation which runs in O(log n)
This repeated k times makes the complexity = O(k log n)

Proof of the Master Method [7]

We use Master Method in solving recurrences in the form:

$T(n) = a\,T(\frac{n}{b}) + f(n)$

Where:

a is number of subproblems
b is factor of dividing the size
f(n) is the work done outside the recursion

---

[4] Wikipedia "Fermat Primality Test" https://en.wikipedia.org/wiki/Fermat_primality_test

[5] Geeks For Geeks "Fermat method of Primality Test"
https://www.geeksforgeeks.org/dsa/fermat-method-of-primality-test/

[6] ChatGPTt for some insights and explanations
[7] Wikipedia "Master theorem (analysis of algorithms)"
https://en.wikipedia.org/wiki/Master_theorem_(analysis_of_algorithms)

Master Theorem gives 3 cases depending on how f(n) compares to: $n^{log_b a}$

Proof Using Recursion Tree:

Shape of the Recursion Tree:

- Root = T(n) = a T($\frac{n}{b}$) + f(n), work at this level = f(n)

The root generates 'a' subproblems of size $\frac{n}{b}$: Each contributes f($\frac{n}{b}$).

- Work at level 1: a f($\frac{n}{b}$)

Each of the 'a' subproblems generates 'a' more: Number of subproblems = a^2 Each has a size $\frac{n}{b^2}$

- Work at level 2: a^2 f($\frac{n}{b^2}$)

Generalizing:

Number of subproblems = a^k

Size of each = $\frac{n}{b^k}$

- Work at level k = a^kf($\frac{n}{b^k}$)

Height of the recursion tree:

- Base Occurs when $\frac{n}{b^k}$ = 1, so b^k = n, k = $log_b n$ (happens at the leaf), so height = $log_b n$

Total Work of the Recursion Tree:

Total Work = Sum of all level

- T(n) = $\sum_{k=0}^{log_b n} a^k$f($\frac{n}{b^k}$)

number of leaves = number of nodes at that level: a^k=$a^{log_b n}$

Simplify using exponent rules: $a^{log_b n}$ = $n^{log_b a}$

This number $n^{log_b a}$ is the total contribution *from the recursive calls*.

Now we want to understand **how large each level is**, and **which level dominates the total**. This is what gives us the 3 cases.

To determine the final asymptotic complexity, we compare: f(n) to $n^{log_b a}$

the relative size of f(n) compared to $n^{log_b a}$ determines which part of the tree dominates.

Case 1: f(n) < $n^{log_b a}$	The leaves dominate because each deeper level contributes more work. T(n) = O($n^{log_b a}$)
Case 2: f(n) = $n^{log_b a}$	Every level contributes the same amount of work. T(n) =O($n^{log_b a}$log n)
Case 3: f(n) < $n^{log_b a}$ + Regularity condition	Then the root (f(n) itself) dominates the overall complexity. T(n) = O(f(n))

Regularity Condition:

The regularity condition ensures that the function f(n) (the non-recursive work) does not grow *too fast* when the problem size shrinks from n to n/b.

Case 3 only works if the work per level decreases as we go down the recursion tree.

If f(n) explodes suddenly when n becomes smaller, the recursion tree becomes unbalanced, and Case 3 fails.

Formula:

a f($\frac{n}{b}$) ≤ c f(n) for some constant c<1

- a f($\frac{n}{b}$) = total work at the next level of the recursion tree
- c f(n) = a fraction of the root work
- The total work of the next level is strictly smaller than the work at the current level.

Fibonacci Series in O(1) [8]

There are many ways to calculate the nth Fibonacci number (Recursion, Matrix Exponentian, Dynamic programming, ..etc)

But none of those actually has a constant O(1) time, the best are the Matrix Exponentian and Fast Doubling Method with O(log n) Complexity, So are there any O(1) fibonacci algorithms? [9]

Actually Yes there is an algorithm that's considered to execute in a constant time which is Binet's Formula or also known as the golden ratio

Formula:

$F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}}$, Where $\phi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$. (the golden ratios)

Since $|\psi| < 1$, for a large n , ψ goes to 0.

But practically this formula isn't used. Why?

- Computer Can not compute ϕ^n for large n:
  - Floating point overflow happens

For large n (e.g., n > 70), floating point becomes inaccurate, because the Fibonacci numbers grow exponentially. You lose many digits and Precision errors accumulate.

This method can calculate the first 34 fibonacci numbers correctly. After that there may be a difference from the correct value.

Instead, the fast doubling method for example is used in python and java.

Binet's Formula proof starts with the recursion $O(2^n)$ approach and optimise it algebraically in various steps until we reach the formula we have.

Proof [10]:

The Fibonacci sequence is defined by the recurrence:

---

[8] Geeks for Geeks "Nth Fibonacci Number" https://www.geeksforgeeks.org/dsa/program-for-nth-fibonacci-number/#other-approach-using-golden-ratio

[9] Geeks for Geeks "Finding nth Fibonacci number using golden ratio" https://www.geeksforgeeks.org/dsa/program-for-nth-fibonacci-number/#other-approach-using-golden-ratio

[10] Medium "Fibonacci in Constant Time" https://medium.com/@adarshtyagi/fibonacci-in-constant-time-945546c9e64c

F(n)=F(n−1)+F(n−2) with initial conditions:

F(0)=0, F(1)=1.

1. We try a solution of the form: $F(n)=\alpha^n$
2. Substitute into the recurrence: $\alpha^n = \alpha^{n-1} + \alpha^{n-2}$
3. Factor by dividing both sides by α^{n-2} (which is non-zero) that will give us: $\alpha^2 = \alpha + 1$
4. Rearrange: $\alpha^2 - \alpha - 1 = 0$
5. Now we solve the quadratic equation that gives us the values of $\phi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$
6. For a recurrence with two distinct roots, the general solution is: $F(n) = C_1\phi^n + C_2\psi^n$
7. Since we know the values of ϕ, ψ, F(0) and F(1) we substitute in the equation to get the known binet's formula

Matrix Multiplication Approaches [11]

There are 2 main approaches or you could say the most famous ones.

1. Classical (Naive) Matrix Multiplication

This is the naive approach of initialising the result matrix with zeros and filling each cell by computing the dot product of the corresponding row from 1st matrix and column from 2nd matrix. This is done using three nested loops: the outer two iterate over the result matrix cells, and the inner most loop performs the multiplication and addition.

Time Complexity:

3 loops with nearly each O(n) or depending on each matrix size so complexity is O(x^3) or more correctly O(n × m × q) for each matrix size.

Code in C++:

```
vector<vector<int>> multiply(const vector<vector<int>> &mat1, const
vector<vector<int>> &mat2)
{
```

[11] Geek for Geeks "Matrix Multiplication" https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/#expected-approach-using-strassens-method-on-3-time-and-on-2-space

```
    int n = mat1.size(), m = mat1[0].size(), q = mat2[0].size();

    vector<vector<int>> res(n, vector<int>(q, 0)); //3rd matrix to hold
result

    //loop through each row of mat1
    for(int i = 0; i < n; i++)
    {

        //loop through each column of mat2
        for(int j = 0; j < q; j++)
        {

            //compute the dot product of
            //row mat1[i] and column mat2[][j]
            for(int k = 0; k < m; k++) res[i][j] += mat1[i][k] * mat2[k][j];

        }

    }

    return res;

}
```

2. Strassen's Matrix Multiplication Algorithm[12]

Strassen algorithm is a divide-and-conquer algorithm for matrix multiplication developed by Volker Strassen.Strassen reduces the number of multiplications by dividing matrices into 4 submatrices.

Naive method multiplies submatrices using **8 multiplications**. Strassen discovered a way to do it using **7 multiplications** instead of 8.

Strassen algorithm cleverly constructs **7 auxiliary matrices**:

$M_1 = (A_{11}+A_{22})(B_{11}+B_{22})$

$M_2 = (A_{21}+A_{22})B_{11}$

$M_3 = A_{11}(B_{12}- B_{22})$

$M_4 = A_{22}(B_{21}-B_{11})$

$M_5 = (A_{11}+A_{12})B_{22}$

$M_6 = (A_{21}-A_{11})(B_{11}+B_{12})$

$M_7 = (A_{12}-A_{22})(B_{21}+B_{22})$

Using these 7 values, we compute the four output blocks:

$C_{11} = M_1 + M_4 - M_5 + M_7$

$C_{12} = M_3 + M_5$

$C_{21} = M_2 + M_4$

[12] Engineering Wing on Youtube "Lec 30: Trick to Remember Strassen's Matrix Multiplication Formulas" https://www.youtube.com/watch?v=HO7tEhH_Dlg&t=81s

$C_{22} = M_1 - M_2 - M_3 + M_6$

Time Complexity:

Since its solved in 7 multiplications not 8 the Relation is: $7T(\frac{n}{2}) + O(n^2)$, when solved by master method gives Time complexity of $O(n^{log_2 7}) \approx O(n^{2.81})$.

Code in C++:

```
//function to Add two matrices
vector<vector<int>> add(const vector<vector<int>> &A, const
vector<vector<int>> &B)
{

   int n = A.size();

   vector<vector<int>> C(n, vector<int>(n)); //matrix to hold result

   for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) C[i][j] = A[i][j] +
B[i][j];

   return C;
}

//function to subtract two matrices
vector<vector<int>> sub(const vector<vector<int>> &A, const
vector<vector<int>> &B)
{

   int n = A.size();

   vector<vector<int>> C(n, vector<int>(n)); //matrix to hold result

   for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) C[i][j] = A[i][j] -
B[i][j];

   return C;

}

//Strassen recursive function
vector<vector<int>> strassen(const vector<vector<int>> &A, const
vector<vector<int>> &B)
{

   int n = A.size();

   //base case: if matrix is 1x1 return direct multiplication
   if(n == 1) return {{A[0][0] * B[0][0]}};

   int k = n / 2; //half size of matrix to divide matrices into 4
sub-matrices

   //creating sub-matrices
   vector<vector<int>> A11(k, vector<int>(k));
   vector<vector<int>> A12(k, vector<int>(k));
   vector<vector<int>> A21(k, vector<int>(k));
```

```
vector<vector<int>> A22(k, vector<int>(k));
vector<vector<int>> B11(k, vector<int>(k));
vector<vector<int>> B12(k, vector<int>(k));
vector<vector<int>> B21(k, vector<int>(k));
vector<vector<int>> B22(k, vector<int>(k));

//splitting matrices into 4 blocks
for(int i = 0; i < k; i++)
{

    for(int j = 0; j < k; j++)
    {

        A11[i][j] = A[i][j];
        A12[i][j] = A[i][j + k];
        A21[i][j] = A[i + k][j];
        A22[i][j] = A[i + k][j + k];

        B11[i][j] = B[i][j];
        B12[i][j] = B[i][j + k];
        B21[i][j] = B[i + k][j];
        B22[i][j] = B[i + k][j + k];

    }

}

//computing the 7 Strassen products recursively
vector<vector<int>> M1 = strassen(add(A11, A22), add(B11, B22));
vector<vector<int>> M2 = strassen(add(A21, A22), B11);
vector<vector<int>> M3 = strassen(A11, sub(B12, B22));
vector<vector<int>> M4 = strassen(A22, sub(B21, B11));
vector<vector<int>> M5 = strassen(add(A11, A12), B22);
vector<vector<int>> M6 = strassen(sub(A21, A11), add(B11, B12));
vector<vector<int>> M7 = strassen(sub(A12, A22), add(B21, B22));

//combine results into C
vector<vector<int>> C(n, vector<int>(n));

vector<vector<int>> C11 = add(sub(add(M1, M4), M5), M7);
vector<vector<int>> C12 = add(M3, M5);
vector<vector<int>> C21 = add(M2, M4);
vector<vector<int>> C22 = add(sub(add(M1, M3), M2), M6);

//placing sub-matrices in result matrix C
for(int i = 0; i < k; i++)
{

    for(int j = 0; j < k; j++)
    {

        C[i][j] = C11[i][j];
        C[i][j + k] = C12[i][j];
        C[i + k][j] = C21[i][j];
        C[i + k][j + k] = C22[i][j];

    }

}

return C;
```

}

3. Which is better and when

The classical matrix multiplication algorithm is better for small and moderately sized matrices because it has no overhead, uses little memory, and is numerically stable. Although it runs in $O(n^3)$, it is usually faster in practice for n below a few hundred.

Strassen's algorithm becomes more efficient when the matrices are large (typically n ≥ 512), because it reduces the number of multiplications and runs in $O(n^{2.81})$. However, it requires more memory, introduces numerical instability, and has higher constant factors due to extra additions. Therefore, Strassen is theoretically faster but only practical for large dense matrices.